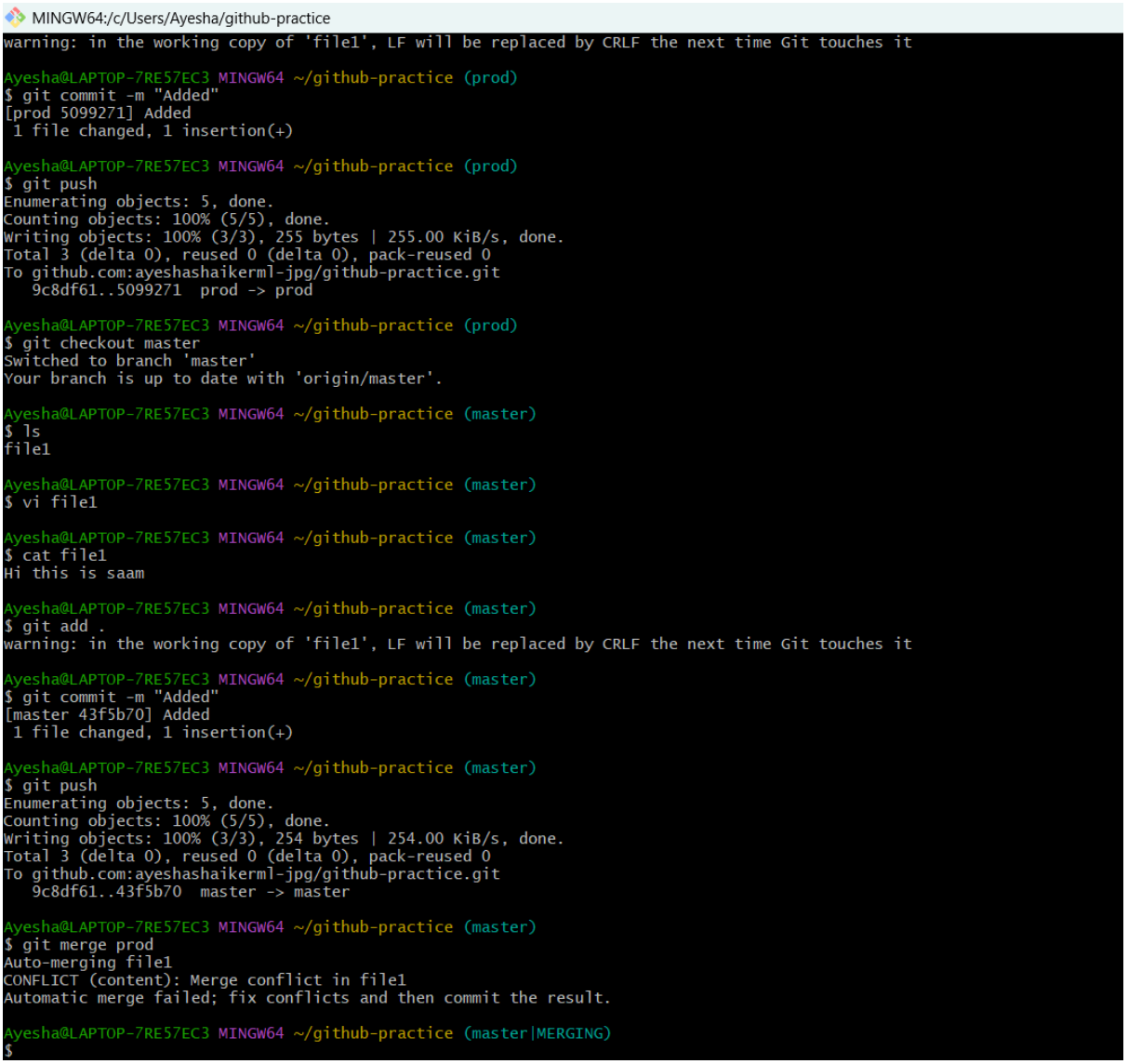
1.Resolve Merge Conflicts

- Create a merge conflict intentionally (two users editing the same line).
- Resolve the conflict and push the changes.



```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git merge prod
Auto-merging file1
CONFLICT (content): Merge conflict in file1
Automatic merge failed; fix conflicts and then commit the result.

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master|MERGING)
$ vi file1

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master|MERGING)
$ git add .

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master|MERGING)
$ git commit -m "Added"
[master 2603a3b] Added

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git push
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Writing objects: 100% (3/3), 297 bytes | 148.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:ayeshashaikerml-jpg/github-practice.git
   43f5b70..2603a3b  master -> master

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git merge prod
Already up to date.

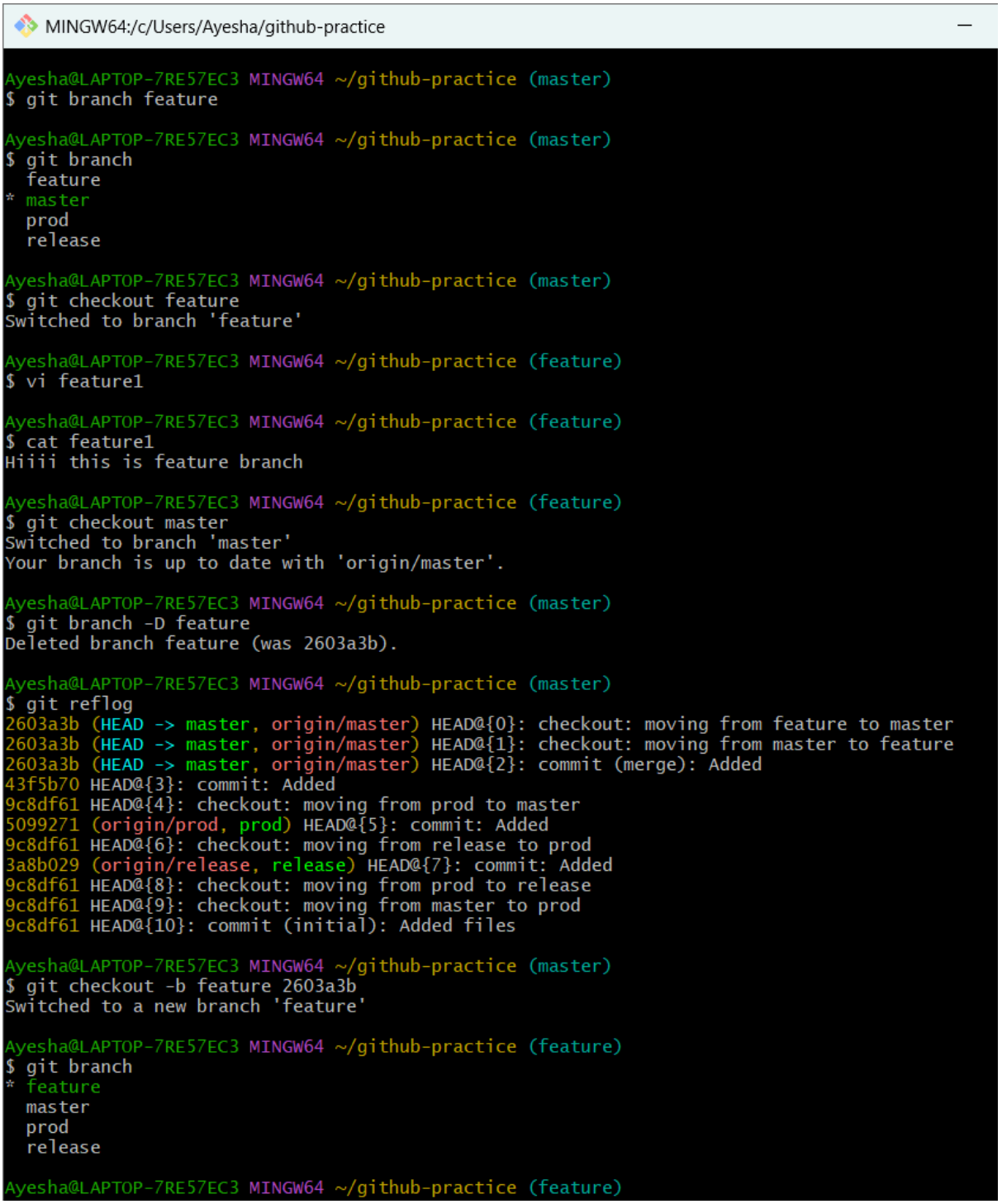
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$
```

Resolve Merge Conflict — My Steps

1. I checked out the master branch because I wanted to merge the changes from the prod branch into master.
2. I merged the prod branch into master.
3. Git detected that both branches had changes to the same content in file1.
4. Git could not automatically decide which change to keep, so it reported a merge conflict in file1.
5. I opened file1 and checked the conflicting section.
6. I saw Git's conflict markers showing:
    - My current master version
    - The changes coming from prod
7. I compared both versions and decided which content should be kept.
8. I removed the conflict markers and kept the correct final content in file1.
9. I saved file1 after resolving the conflict.
10. I staged the resolved file to tell Git that I had fixed the conflict.
11. I completed the merge by creating the merge commit.
12. I pushed the updated master branch to the remote GitHub repository.
13. Finally, I verified the GitHub repository and confirmed that the resolved version of file1 was present.

2. Recover Deleted Branch

- Delete a local branch and then recover it using the reflog.



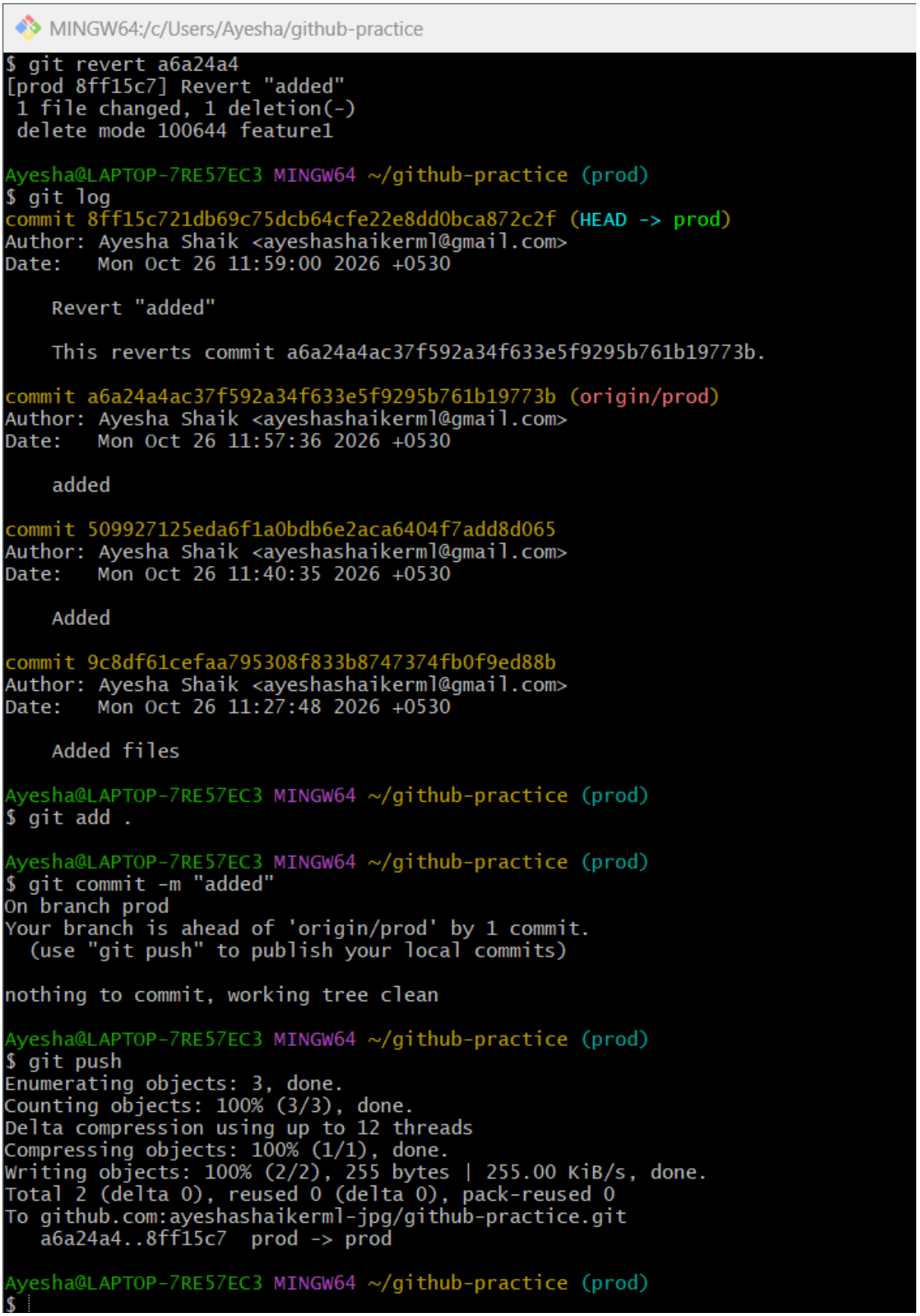
Recover Deleted Branch — Steps

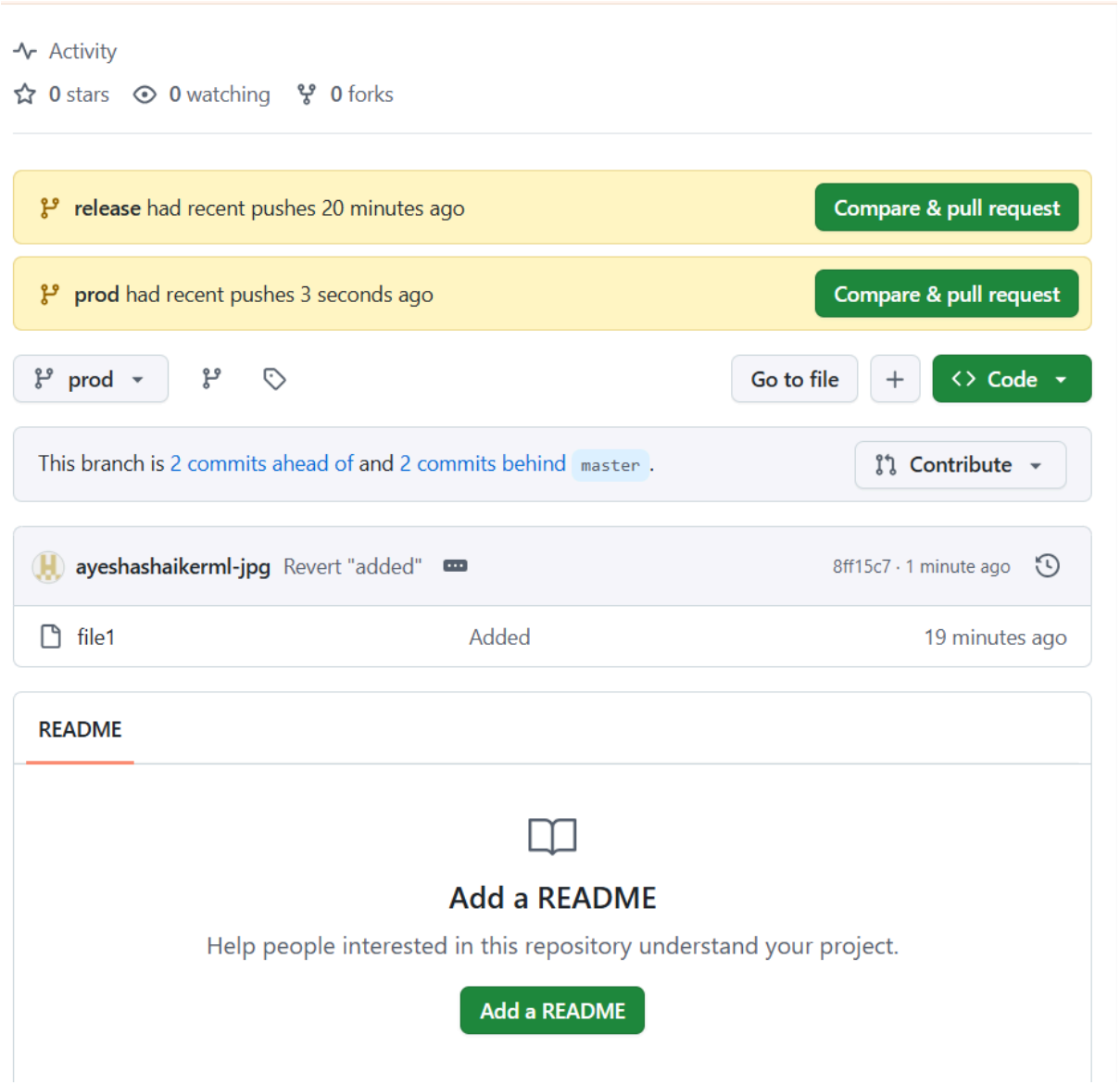
1. I created a branch named feature for practice.
2. I made some changes in the feature branch.
3. I committed the changes so that the branch has commit history.
4. I switched to another branch, such as master.

5. I deleted the local feature branch intentionally.
6. The feature branch is now deleted from my local repository.
7. I checked the Git reflog to find the commit that was previously pointed to by the deleted feature branch.
8. I identified the latest commit ID belonging to the deleted feature branch.
9. I recreated the feature branch using that commit ID from the reflog.
10. I verified that the feature branch was recovered successfully.
11. I switched to the recovered feature branch.
12. I verified that my previous committed changes were still available.
13. If required, I pushed the recovered branch to GitHub to restore it on the remote repository as well.

3. Undo Wrong Push

Push a wrong commit to GitHub, then undo it without losing history.


MINGW64:/c/Users/Ayesha/github-practice
$ git revert a6a24a4
[prod 8ff15c7] Revert "added"
 1 file changed, 1 deletion(-)
 delete mode 100644 feature1
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (prod)
$ git log
commit 8ff15c721db69c75dcb64cfe22e8dd0bca872c2f (HEAD -> prod)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:59:00 2026 +0530
    Revert "added"
    This reverts commit a6a24a4ac37f592a34f633e5f9295b761b19773b.
commit a6a24a4ac37f592a34f633e5f9295b761b19773b (origin/prod)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:57:36 2026 +0530
    added
commit 509927125eda6f1a0bdb6e2aca6404f7add8d065
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:40:35 2026 +0530
    Added
commit 9c8df61cefaa795308f833b8747374fb0f9ed88b
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:27:48 2026 +0530
    Added files
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (prod)
$ git add .
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (prod)
$ git commit -m "added"
On branch prod
Your branch is ahead of 'origin/prod' by 1 commit.
  (use "git push" to publish your local commits)
nothing to commit, working tree clean
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (prod)
$ git push
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (1/1), done.
Writing objects: 100% (2/2), 255 bytes | 255.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:ayeshashaikerml-jpg/github-practice.git
   a6a24a4..8ff15c7 prod -> prod
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (prod)
$



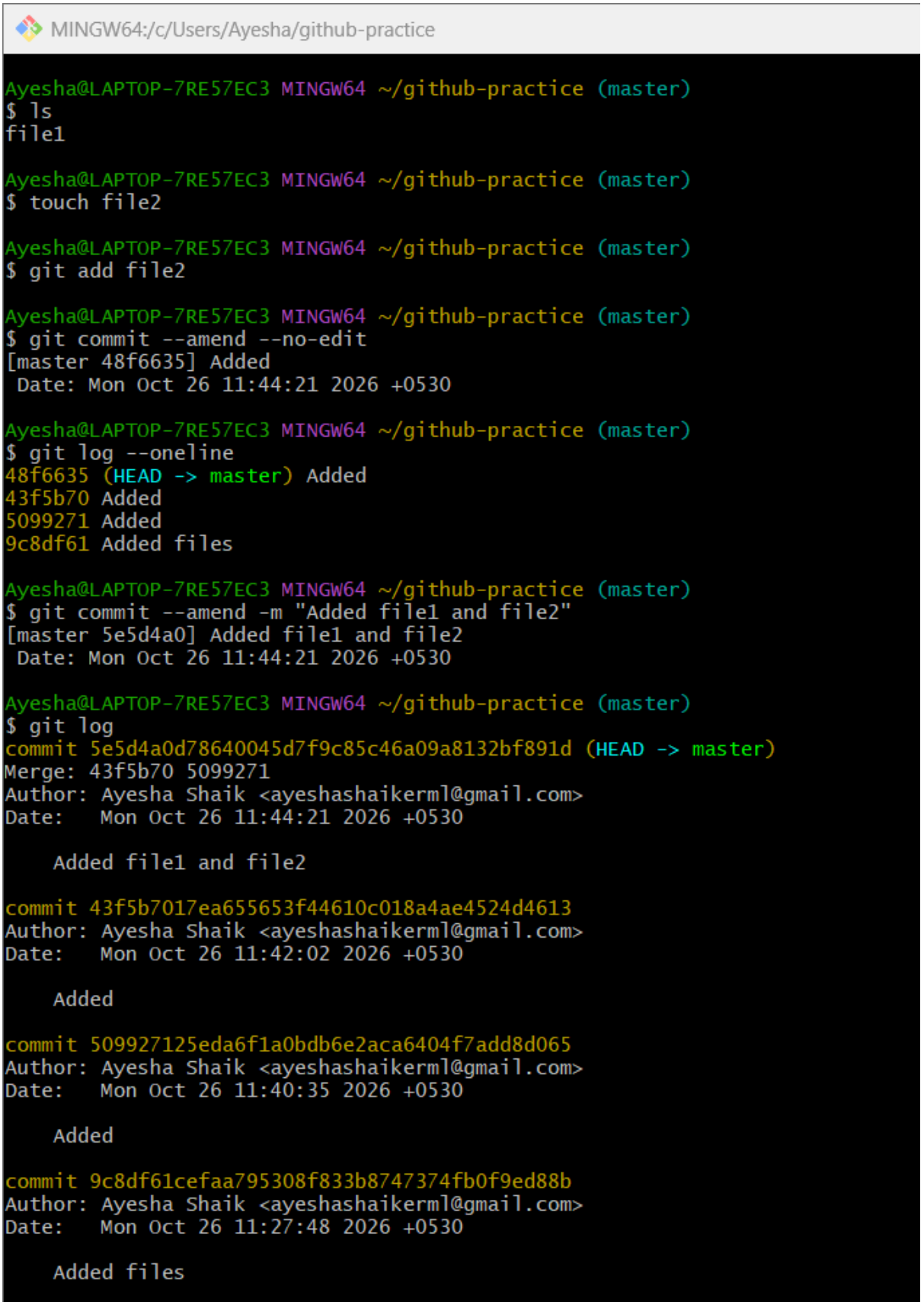
Push Wrong Commit to GitHub and Undo It — Steps

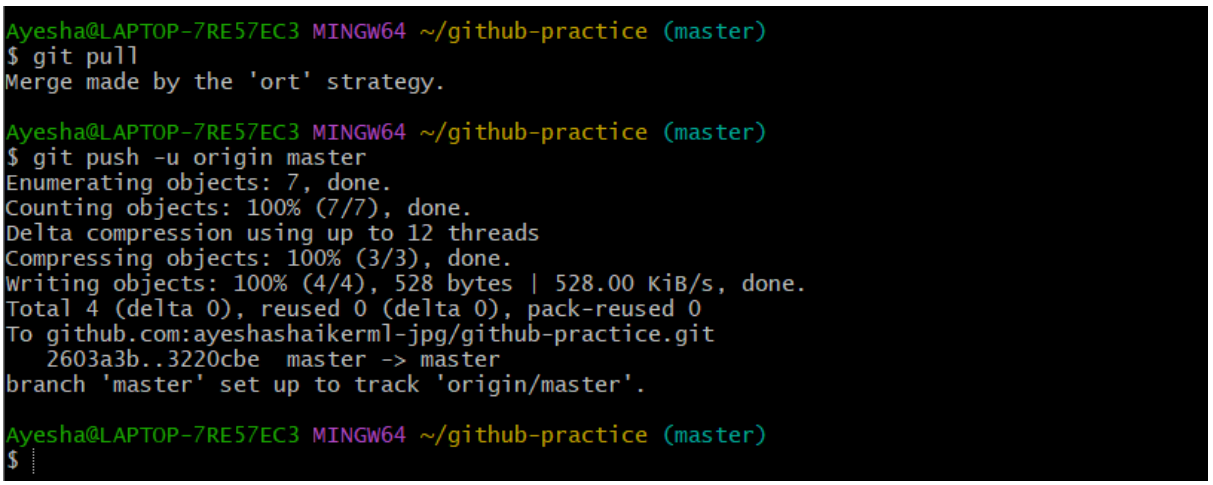
1. I made an incorrect change in feature1.
2. I committed the incorrect change with the message added.
3. I pushed the commit to the prod branch on GitHub.
4. I realized that the commit was incorrect and identified its commit ID: a6a24a4...
5. I reverted that commit. Git created a new commit called Revert "added".
6. The original wrong commit was not deleted from the history.
7. I checked the Git history and confirmed that both commits were present:
    - a6a24a4... → original added commit
    - 8ff15c7... → Revert "added" commit
8. I checked the working tree and confirmed that there were no additional changes to commit.
9. I pushed the new revert commit to GitHub.

10. I verified the GitHub repository and confirmed that the wrong change was undone while the complete commit history was preserved.

4. Amend a Commit

Make a commit, then add a missing file to it using git commit --amend.


MINGW64:/c/Users/Ayesha/github-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ ls
file1
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ touch file2
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git add file2
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git commit --amend --no-edit
[master 48f6635] Added
Date: Mon Oct 26 11:44:21 2026 +0530
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git log --oneline
48f6635 (HEAD -> master) Added
43f5b70 Added
5099271 Added
9c8df61 Added files
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git commit --amend -m "Added file1 and file2"
[master 5e5d4a0] Added file1 and file2
Date: Mon Oct 26 11:44:21 2026 +0530
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (master)
$ git log
commit 5e5d4a0d78640045d7f9c85c46a09a8132bf891d (HEAD -> master)
Merge: 43f5b70 5099271
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:44:21 2026 +0530
Added file1 and file2
commit 43f5b7017ea655653f44610c018a4ae4524d4613
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:42:02 2026 +0530
Added
commit 509927125eda6f1a0bdb6e2aca6404f7add8d065
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:40:35 2026 +0530
Added
commit 9c8df61cefaa795308f833b8747374fb0f9ed88b
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:27:48 2026 +0530
Added files

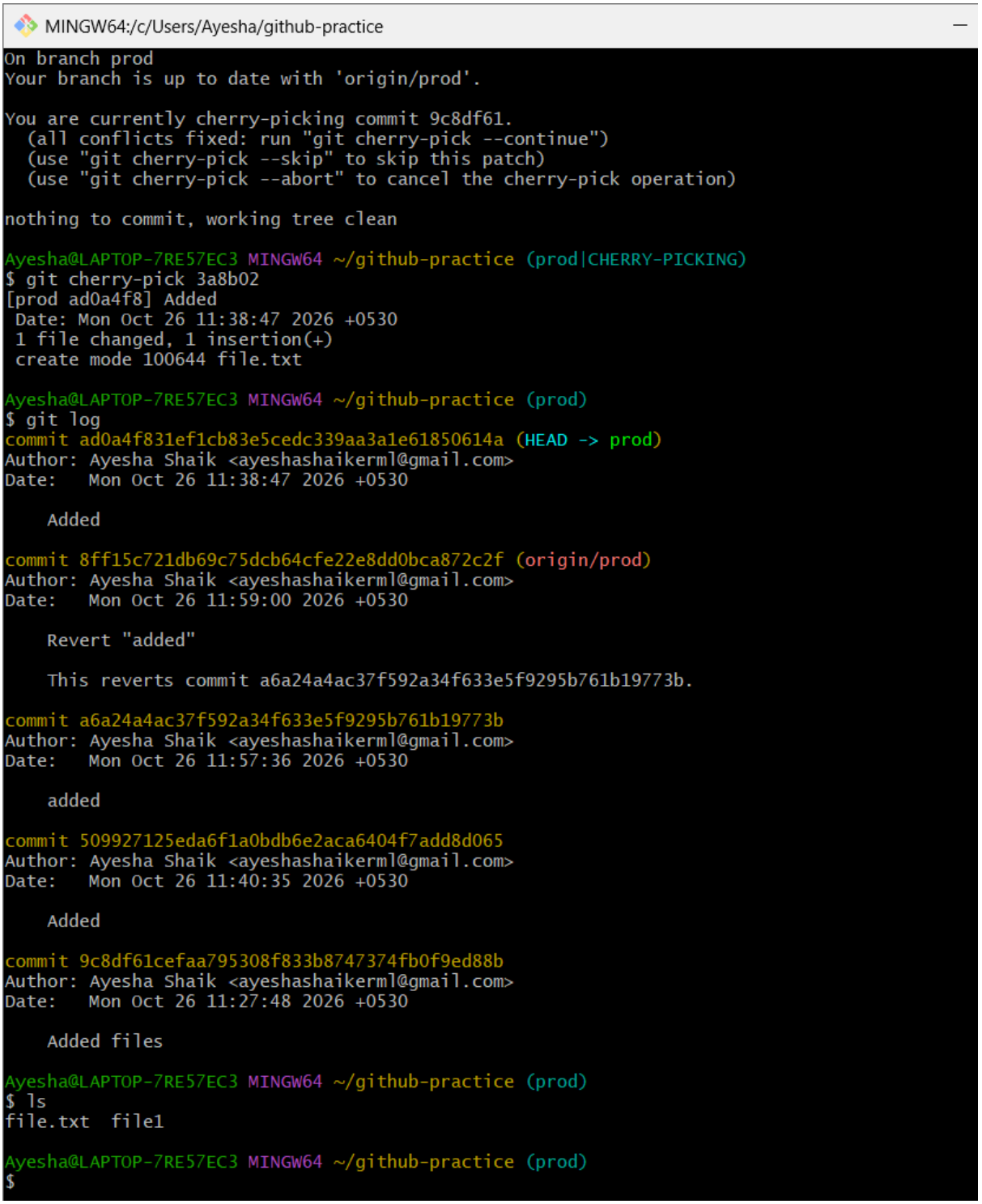


Steps

1. I create file1 and add some content.
2. I stage file1.
3. I commit file1 with a message such as Added files.
4. I realize that I forgot to include file2 in the commit.
5. I create file2 and add the required content.
6. I stage file2.
7. Instead of creating another commit, I amend the previous commit using git commit --amend.
8. Git updates the previous commit and includes file2 in it.
9. I can also modify the previous commit message during the amend if required.
10. I verify the Git history and confirm that there is one commit containing both file1 and file2.

5. Cherry-pick a Commit

Take a specific commit from one branch and apply it to another branch.



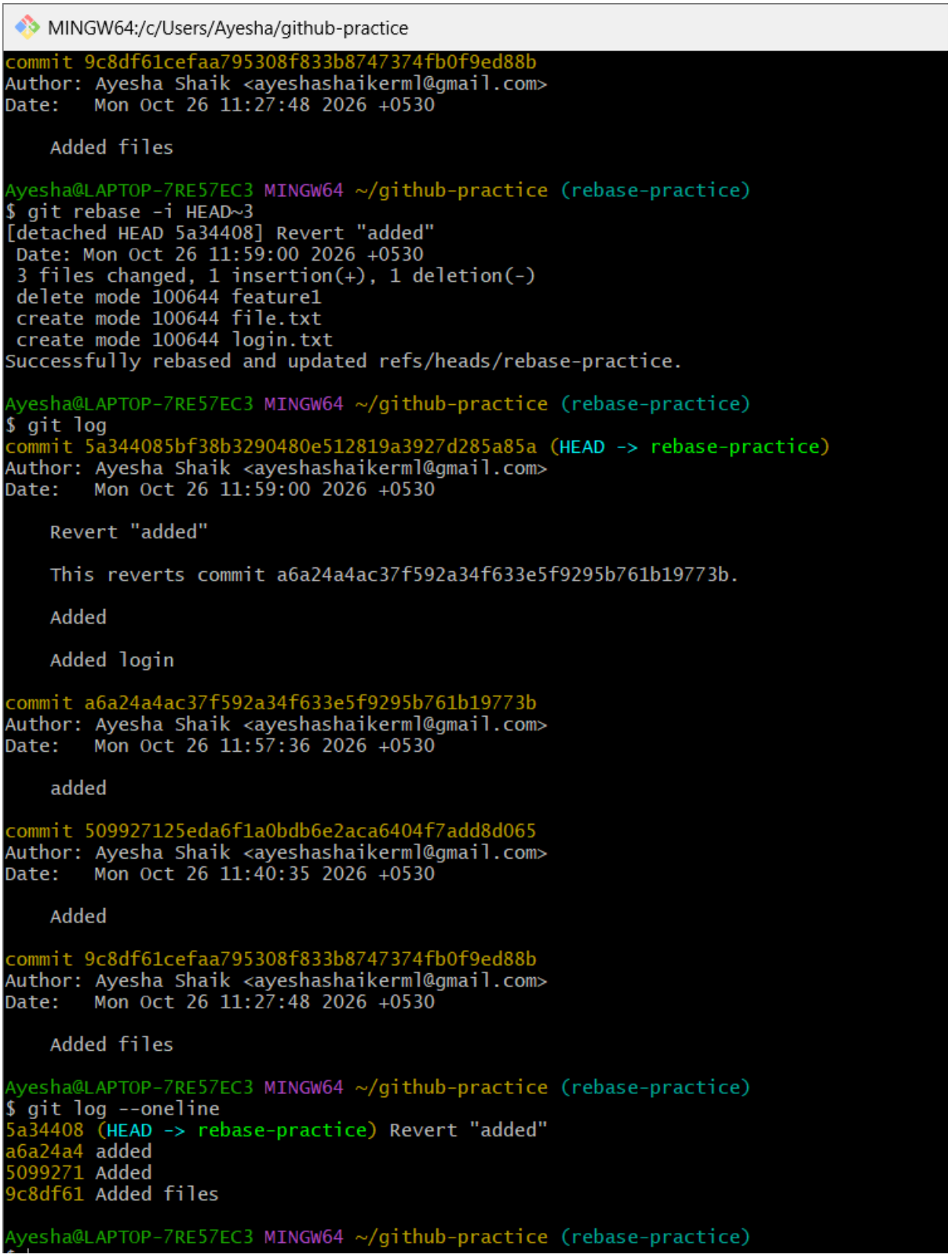
Cherry-Pick — Steps

1. I switched to the prod branch where I wanted to bring a specific commit.
1. I started a cherry-pick operation for a commit from another branch.
2. Git showed that I was already in a cherry-picking state and that there were no conflicts to resolve.

3. I selected the specific commit 3a8b02 that I wanted to bring into the prod branch.
4. Git applied the changes from that commit to the prod branch.
5. Git created a new commit on prod with the same changes.
6. I checked the Git history and confirmed that the new cherry-picked commit appeared at the top of the prod branch.
7. I verified that the required file, file.txt, was present in the working directory.
8. The cherry-pick operation was completed successfully, and the working tree was clean.

6. Interactive Rebase

Reorder and squash multiple commits into a single clean commit.



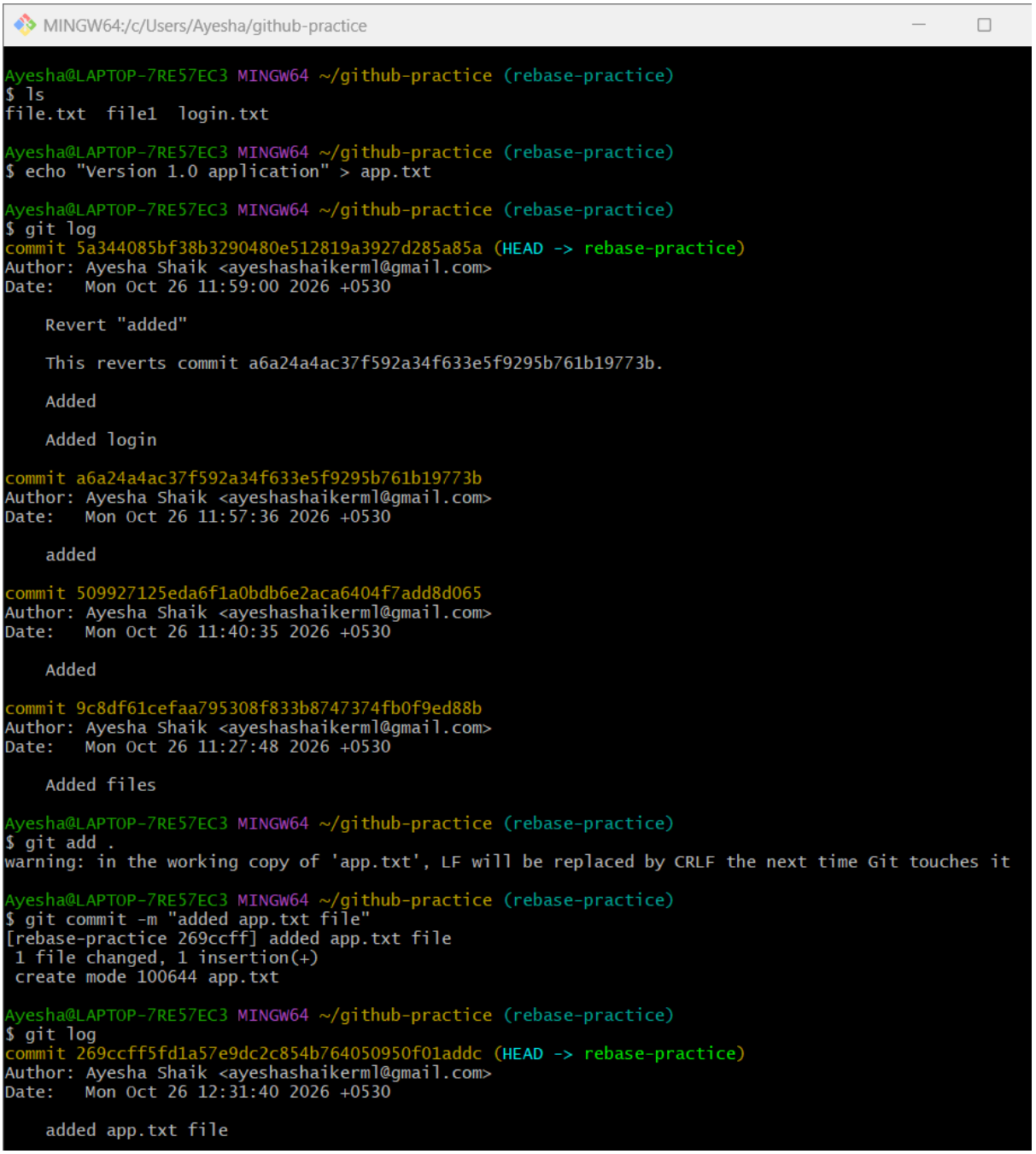
Interactive Rebase — Steps

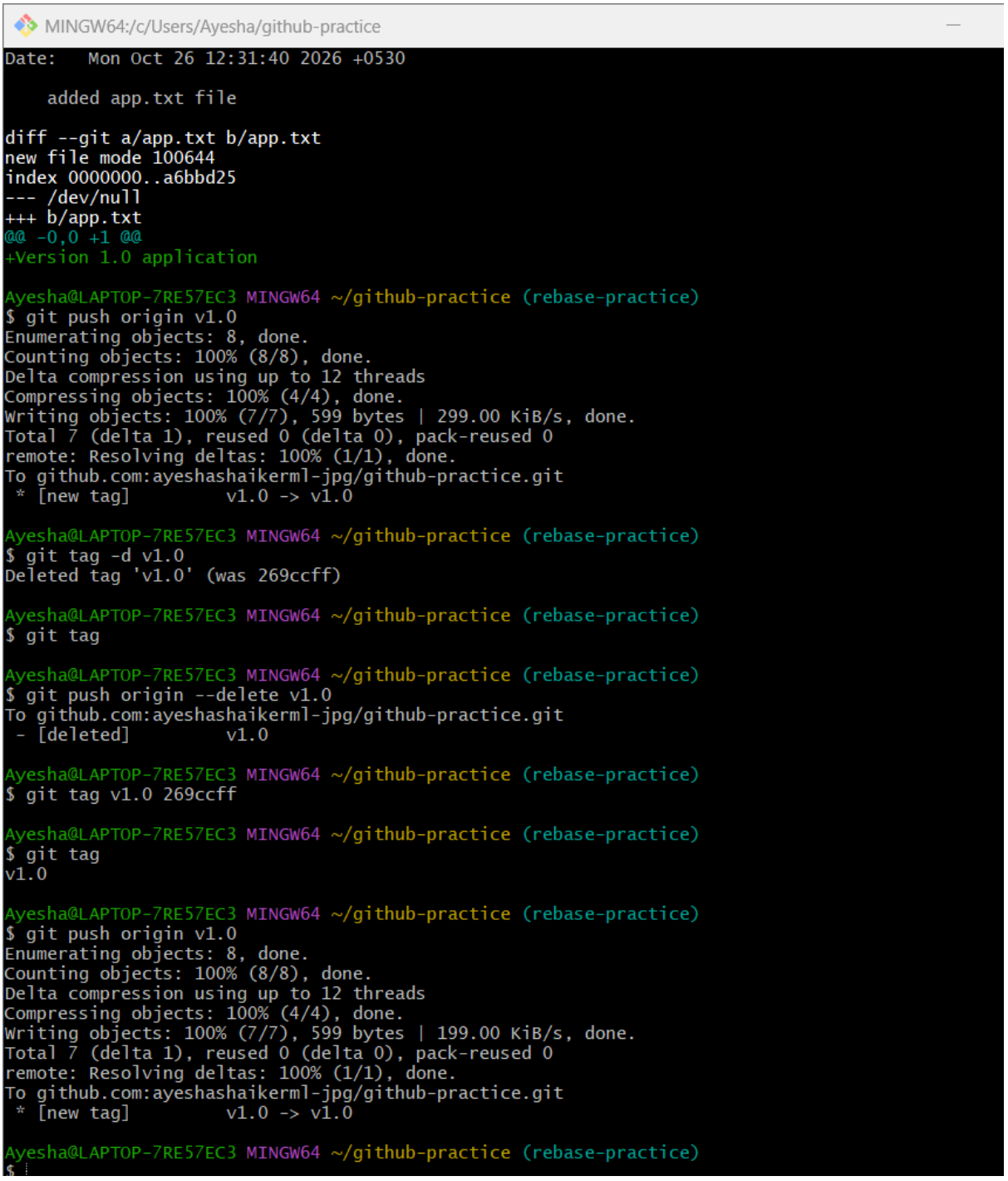
1. I create a new practice branch from my current branch.
2. I create the first commit with some changes.

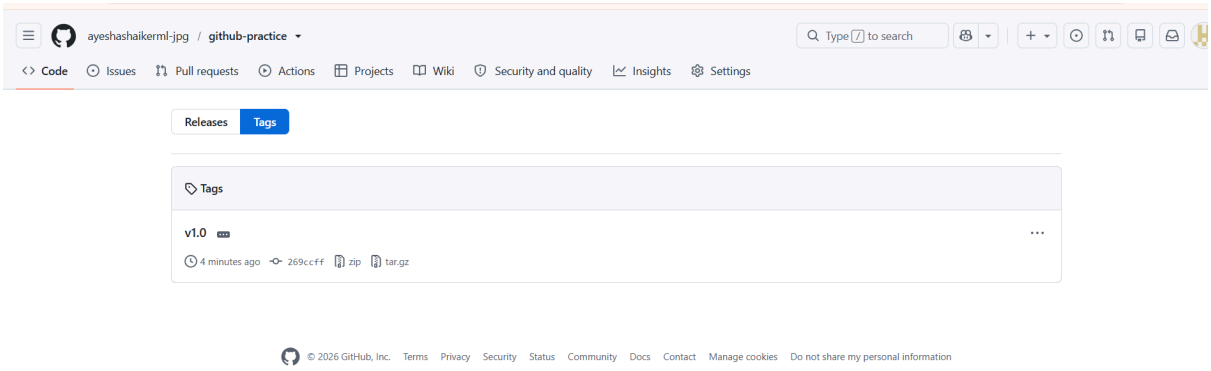
3. I create the second commit with another related change.
4. I create the third commit with another related change.
5. I check the history and confirm that I have three separate commits.
6. I start an interactive rebase for those three recent commits.
7. Git opens the list of the three commits with pick beside each one.
8. I keep pick for the first commit.
9. I change the second commit from pick to squash.
10. I change the third commit from pick to squash.
11. I save and close the rebase instructions.
12. Git combines the second and third commits into the first commit.
13. Git asks me to review the combined commit message.
14. I keep or edit the message to make it clean and meaningful.
15. I save and close the commit message.
16. Git completes the interactive rebase.
17. I check the Git history and confirm that the three commits have become one clean commit.

7. Tagging & Release

Create a version tag (v1.0), push it to GitHub, then delete and restore it.


MINGW64:/c/Users/Ayesha/github-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ ls
file.txt file1 login.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ echo "Version 1.0 application" > app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git log
commit 5a344085bf38b3290480e512819a3927d285a85a (HEAD -> rebase-practice)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:59:00 2026 +0530
Revert "added"
This reverts commit a6a24a4ac37f592a34f633e5f9295b761b19773b.
Added
Added login
commit a6a24a4ac37f592a34f633e5f9295b761b19773b
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:57:36 2026 +0530
added
commit 509927125eda6f1a0bdb6e2aca6404f7add8d065
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:40:35 2026 +0530
Added
commit 9c8df61cefaa795308f833b8747374fb0f9ed88b
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 11:27:48 2026 +0530
Added files
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git add .
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git commit -m "added app.txt file"
[rebase-practice 269ccff] added app.txt file
 1 file changed, 1 insertion(+)
 create mode 100644 app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git log
commit 269ccff5fd1a57e9dc2c854b764050950f01addc (HEAD -> rebase-practice)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 12:31:40 2026 +0530
added app.txt file


MINGW64:/c/Users/Ayesha/github-practice
Date: Mon Oct 26 12:31:40 2026 +0530
added app.txt file
diff --git a/app.txt b/app.txt
new file mode 100644
index 0000000..a6bbd25
--- /dev/null
+++ b/app.txt
@@ -0,0 +1 @@
+Version 1.0 application
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git push origin v1.0
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 599 bytes | 299.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:ayeshashaikerml-jpg/github-practice.git
* [new tag]   v1.0 -> v1.0
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 269ccff)
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git tag
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git push origin --delete v1.0
To github.com:ayeshashaikerml-jpg/github-practice.git
- [deleted]   v1.0
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git tag v1.0 269ccff
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git tag
v1.0
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$ git push origin v1.0
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 599 bytes | 199.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), done.
To github.com:ayeshashaikerml-jpg/github-practice.git
* [new tag]   v1.0 -> v1.0
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (rebase-practice)
$

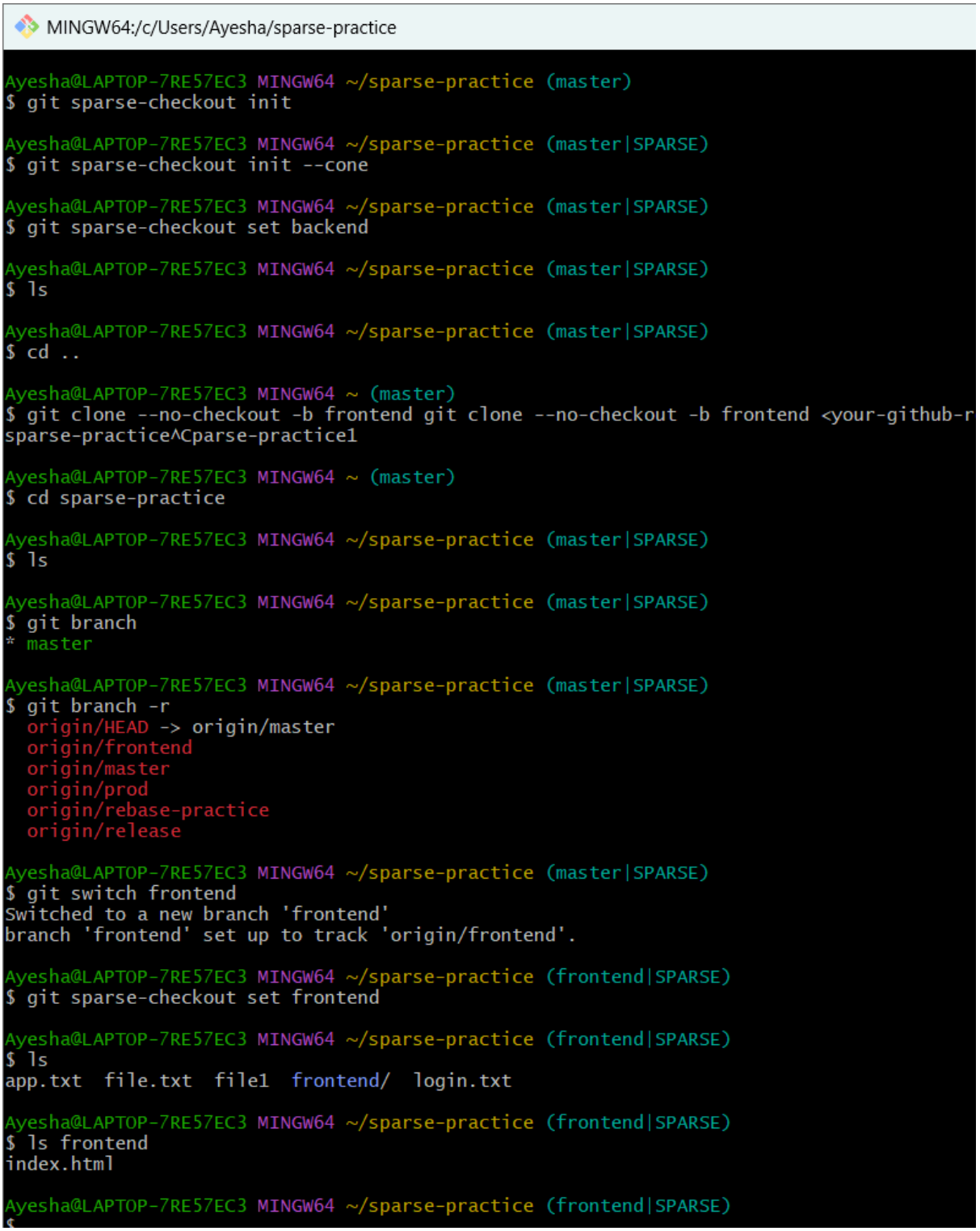


Tagging & Release — v1.0

1. I first make sure I am on the branch containing the release version.
2. I create a tag named v1.0 on the current release commit.
3. I verify that the v1.0 tag exists locally.
4. I push the v1.0 tag to GitHub.
5. I open GitHub and verify that v1.0 is visible under Tags/Releases.
6. To practice deletion, I delete the v1.0 tag from my local repository.
7. I verify that the local v1.0 tag is removed.
8. I also delete the v1.0 tag from GitHub.
9. I verify on GitHub that v1.0 is no longer available.
10. I recreate the v1.0 tag on the same release commit.
11. I push the recreated v1.0 tag to GitHub.
12. I verify that v1.0 has been restored successfully.

8. Clone with Sparse Checkout

Clone only a subdirectory of a repo using sparse checkout.



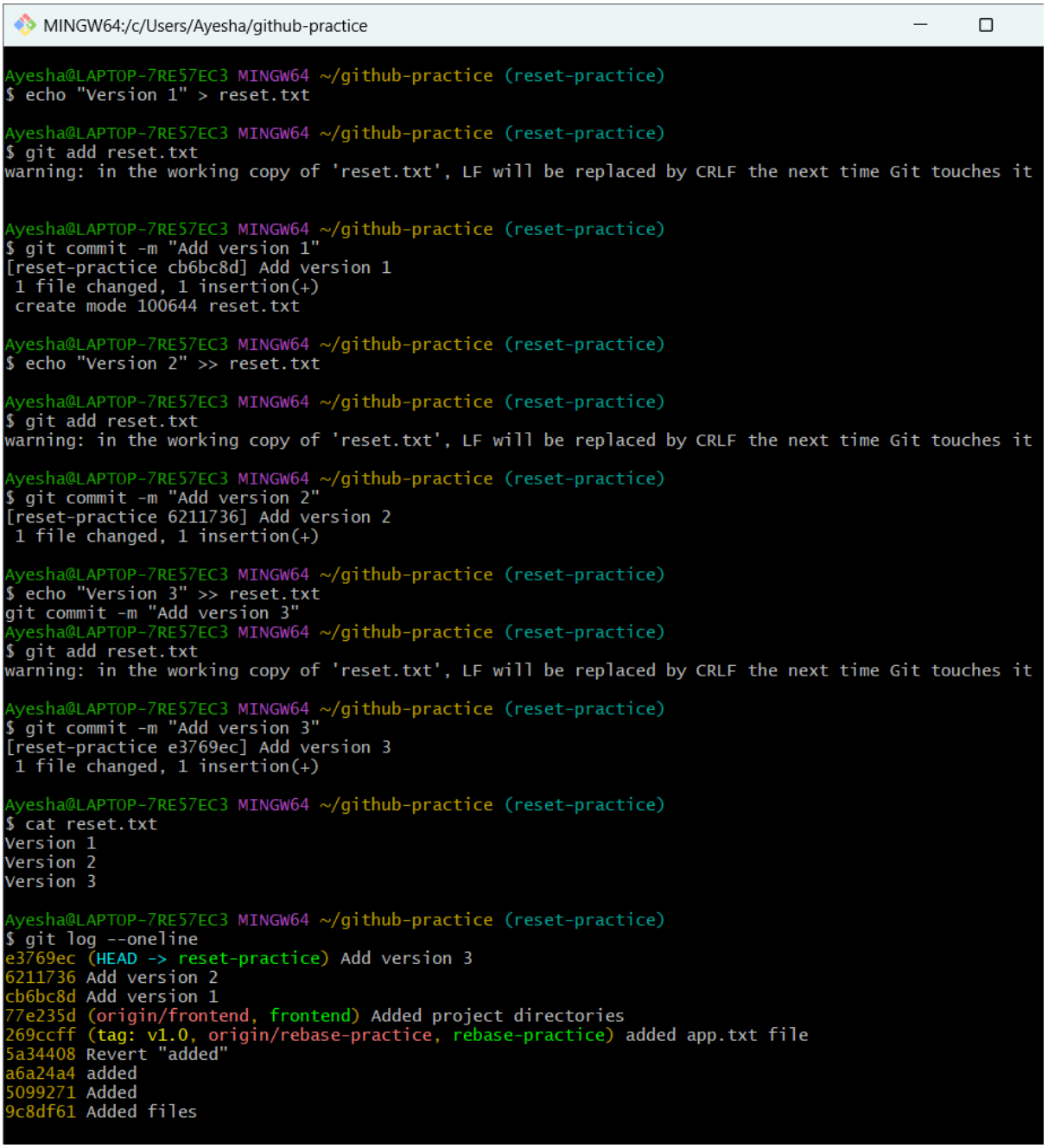
Sparse Checkout — Steps

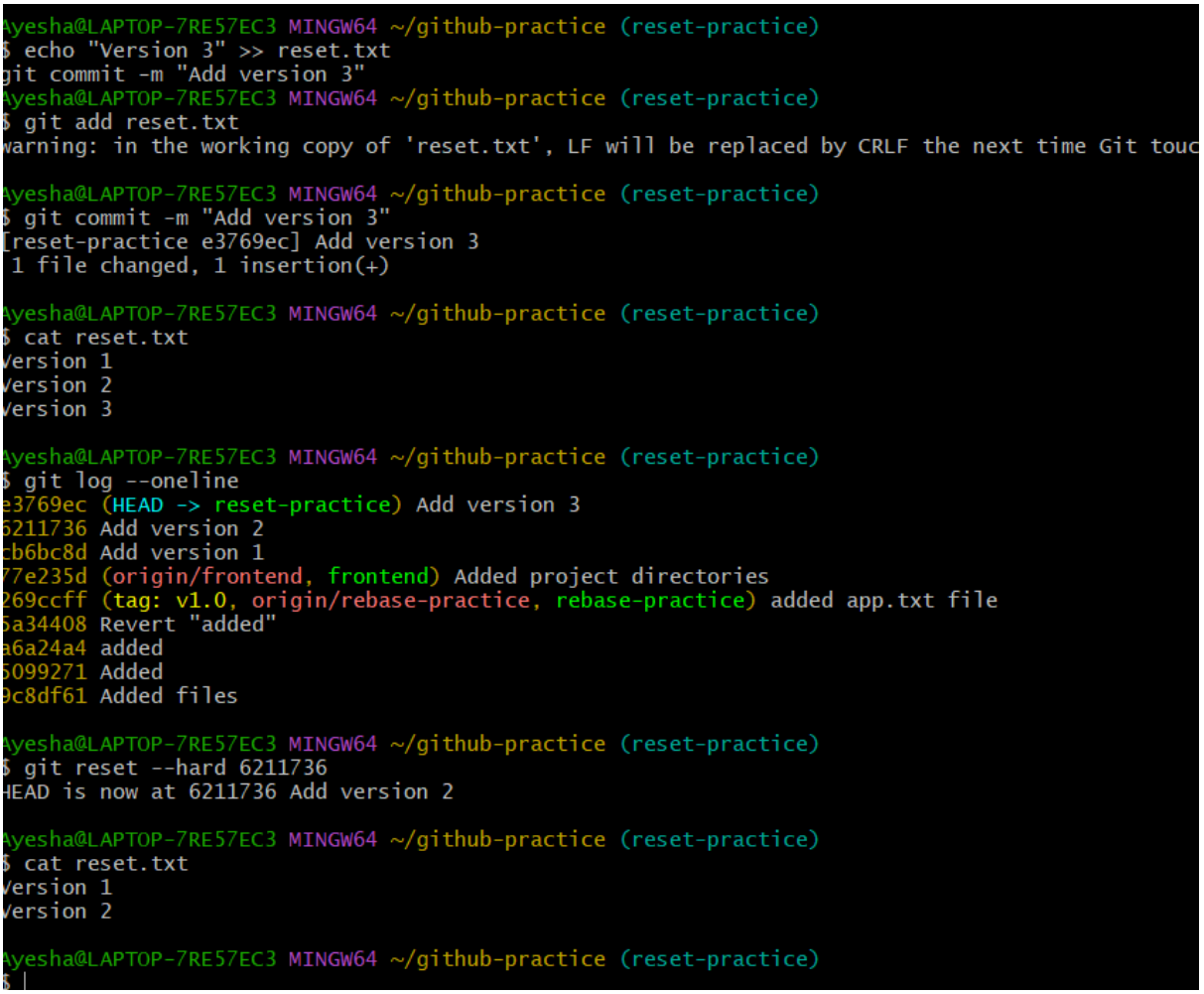
1. From ~/sparse-practice, check the remote branches.
2. Switch to the frontend branch.
3. Enable sparse checkout.
4. Select only the frontend directory.

5. Run ls and verify that only frontend is visible.
6. Open the frontend directory and verify that index.html is present.
7. Do not modify your original ~/github-practice repository; use ~/sparse-practice for this practice.

9. Reset vs Revert Challenge

Demonstrate the difference between git reset --hard and git revert in a repo.


MINGW64:/c/Users/Ayesha/github-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ echo "Version 1" > reset.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git add reset.txt
warning: in the working copy of 'reset.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git commit -m "Add version 1"
[reset-practice cb6bc8d] Add version 1
1 file changed, 1 insertion(+)
create mode 100644 reset.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ echo "Version 2" >> reset.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git add reset.txt
warning: in the working copy of 'reset.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git commit -m "Add version 2"
[reset-practice 6211736] Add version 2
1 file changed, 1 insertion(+)
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ echo "Version 3" >> reset.txt
git commit -m "Add version 3"
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git add reset.txt
warning: in the working copy of 'reset.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git commit -m "Add version 3"
[reset-practice e3769ec] Add version 3
1 file changed, 1 insertion(+)
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ cat reset.txt
Version 1
Version 2
Version 3
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (reset-practice)
$ git log --oneline
e3769ec (HEAD -> reset-practice) Add version 3
6211736 Add version 2
cb6bc8d Add version 1
77e235d (origin/frontend, frontend) Added project directories
269ccff (tag: v1.0, origin/rebase-practice, rebase-practice) added app.txt file
5a34408 Revert "added"
a6a24a4 added
5099271 Added
9c8df61 Added files



Part 1 — git reset --hard

1. Create a practice branch

git checkout -b reset-practice

2. Create the first version

echo "Version 1" > reset.txt
git add reset.txt
git commit -m "Add version 1"

Now:

Add version 1
↑
HEAD

3. Make a second commit

echo "Version 2" >> reset.txt

git add reset.txt
git commit -m "Add version 2"

Now the file contains:

Version 1
Version 2

4. Make a third commit

echo "Version 3" >> reset.txt
git add reset.txt
git commit -m "Add version 3"

Now:

Version 1
Version 2
Version 3

History:

Commit 1 → Commit 2 → Commit 3
↑
HEAD

5. Check the history

git log --oneline

You will see something similar to:

c3c3c3c Add version 3
b2b2b2b Add version 2
a1a1a1a Add version 1

6. Reset back to Commit 2

Copy the Commit 2 ID and run:

git reset --hard b2b2b2b

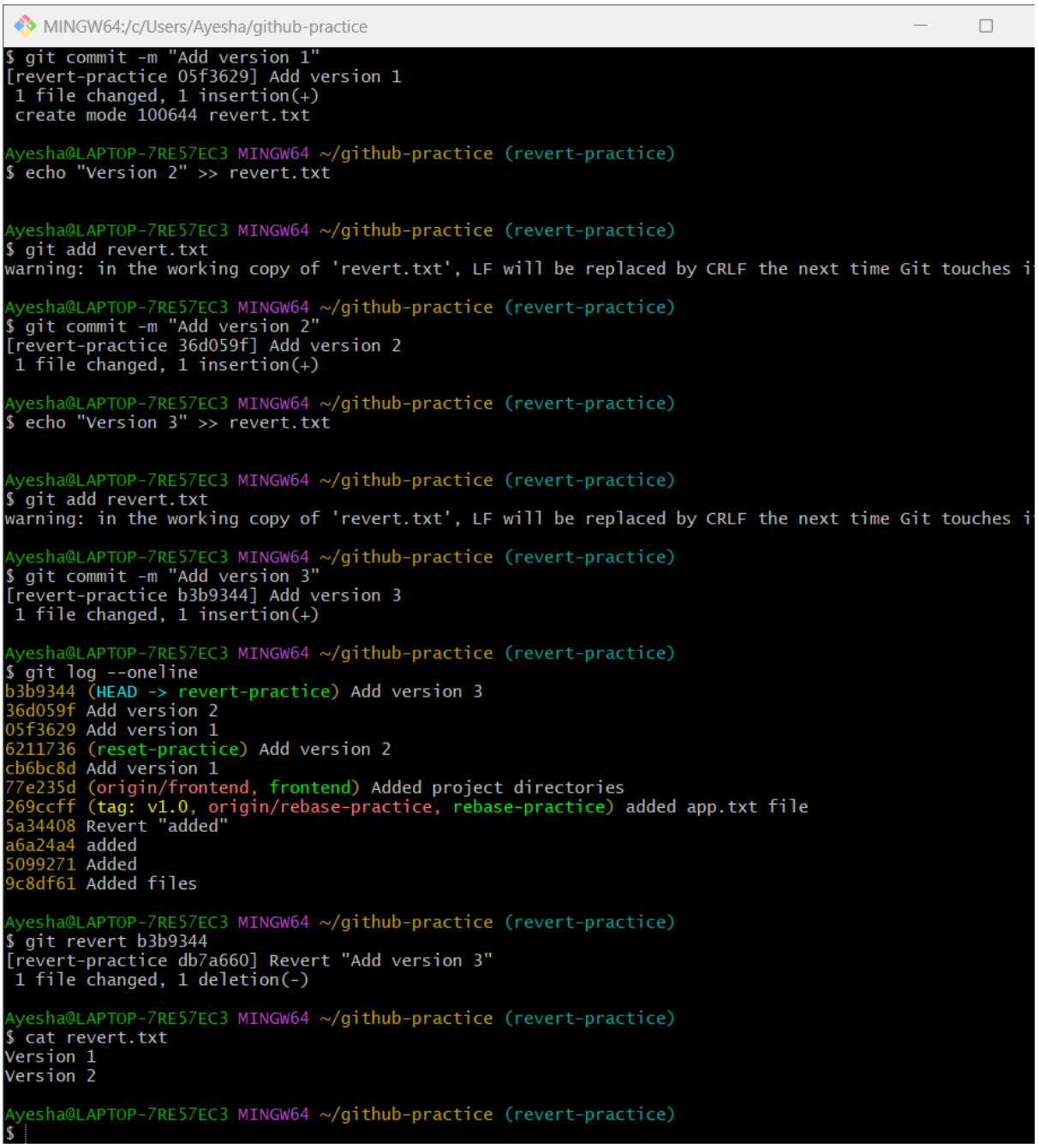
7. Check the file

cat reset.txt

You should now get:

Version 1
Version 2

Version 3 is gone from the current branch.



1. Create another branch

First go back to a clean starting point:

git checkout -b revert-practice

2. Create Version 1

echo "Version 1" > revert.txt

```
git add revert.txt
git commit -m "Add version 1"
```

3. Create Version 2

```
echo "Version 2" >> revert.txt
git add revert.txt
git commit -m "Add version 2"
```

4. Create Version 3

```
echo "Version 3" >> revert.txt
git add revert.txt
git commit -m "Add version 3"
```

Your file:

Version 1
Version 2
Version 3

History:

Commit 1 → Commit 2 → Commit 3
↑
HEAD

5. Find Commit 3

```
git log --oneline
```

For example:

c3c3c3c Add version 3
b2b2b2b Add version 2
a1a1a1a Add version 1

6. Revert Commit 3

```
git revert c3c3c3c
```

Git may open an editor for the revert commit message. Save and close it.

7. Check the file

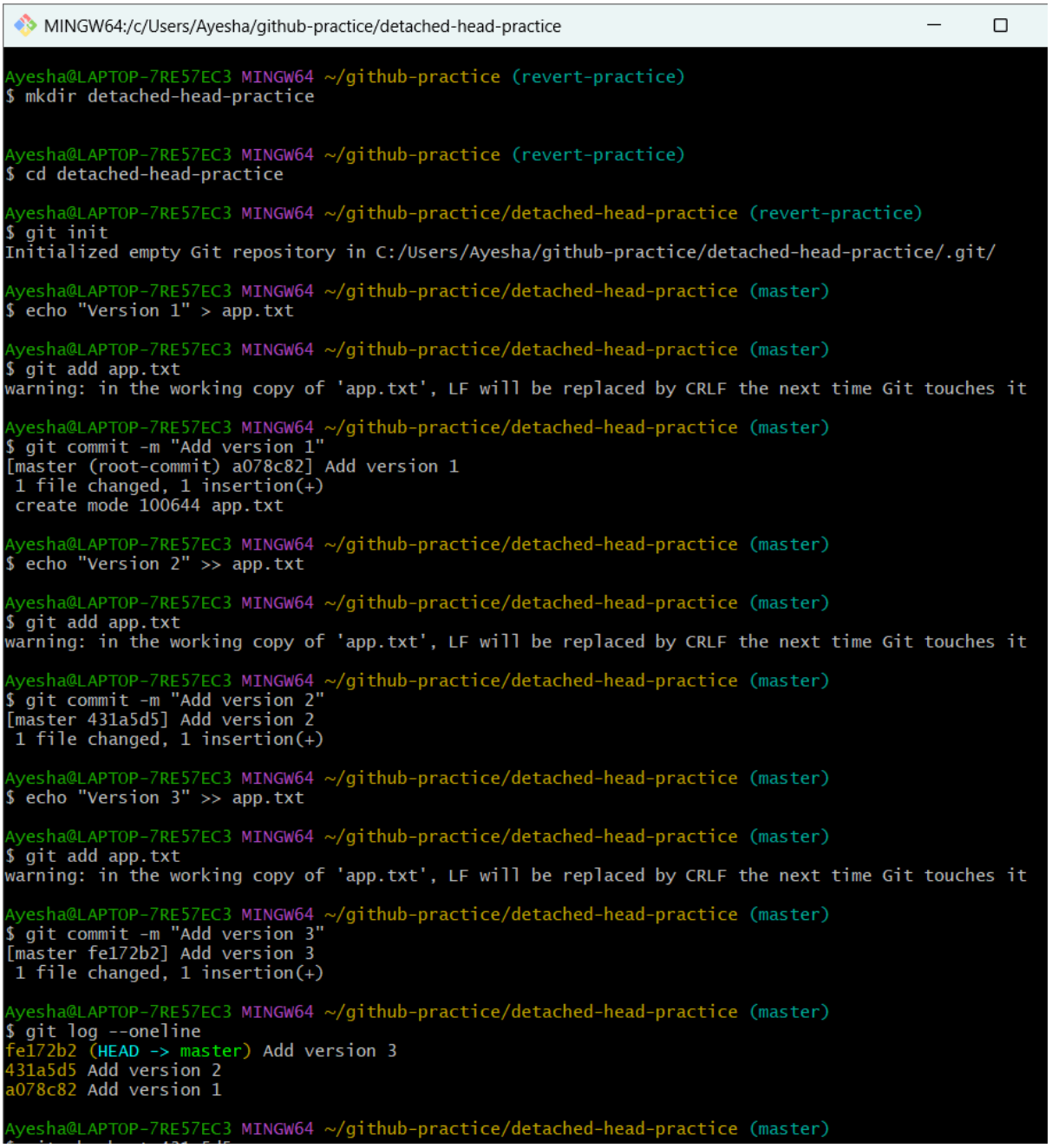
```
cat revert.txt
```

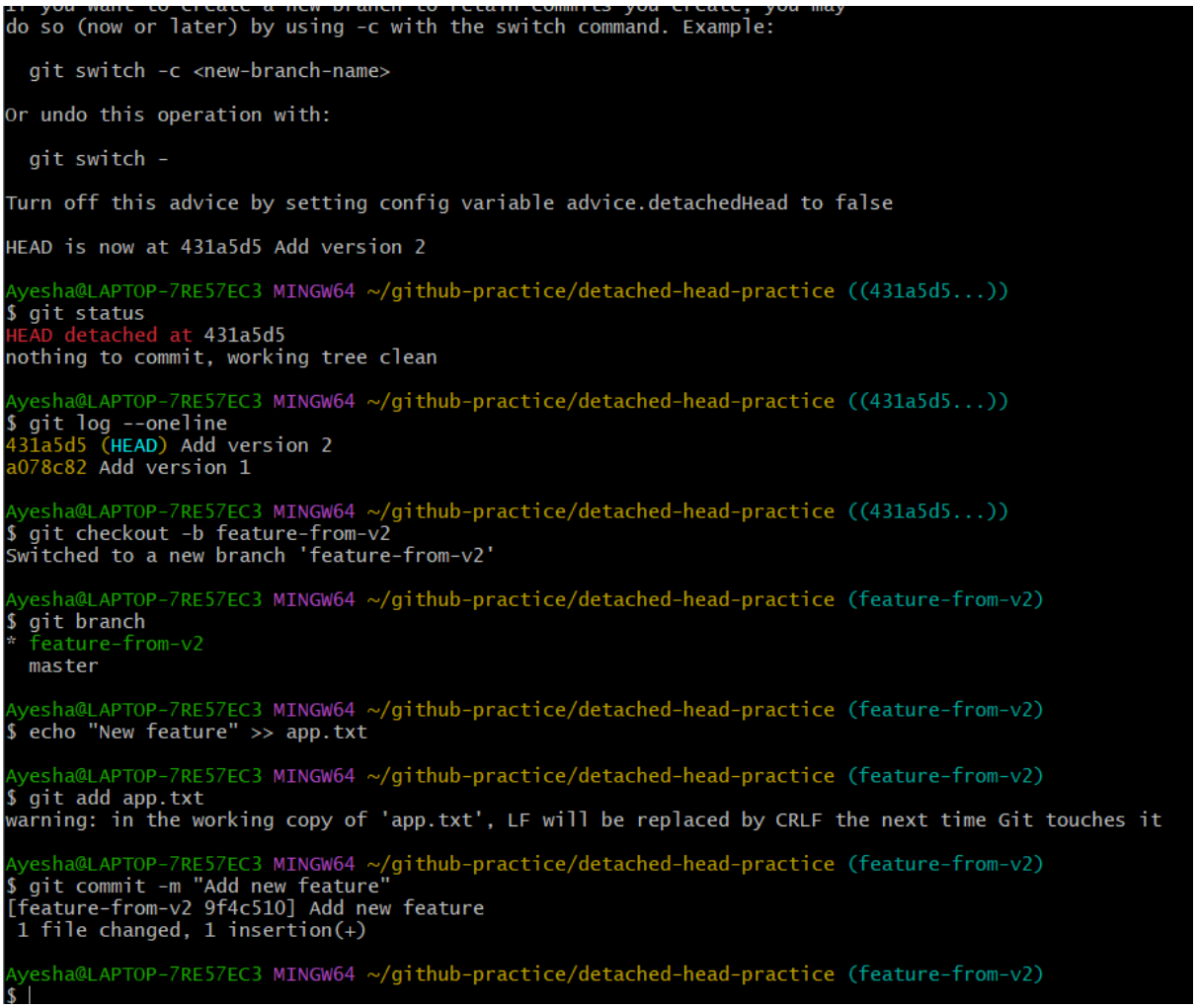
You should get:

Version 1
Version 2

So Version 3's changes have been undone.

10. Detached HEAD Challenge
Checkout a specific commit (detached HEAD state) and create a new branch from it.


MINGW64:/c/Users/Ayesha/github-practice/detached-head-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (revert-practice)
$ mkdir detached-head-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (revert-practice)
$ cd detached-head-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (revert-practice)
$ git init
Initialized empty Git repository in C:/Users/Ayesha/github-practice/detached-head-practice/.git/
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ echo "Version 1" > app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git commit -m "Add version 1"
[master (root-commit) a078c82] Add version 1
1 file changed, 1 insertion(+)
create mode 100644 app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ echo "Version 2" >> app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git commit -m "Add version 2"
[master 431a5d5] Add version 2
1 file changed, 1 insertion(+)
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ echo "Version 3" >> app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git commit -m "Add version 3"
[master fe172b2] Add version 3
1 file changed, 1 insertion(+)
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)
$ git log --oneline
fe172b2 (HEAD -> master) Add version 3
431a5d5 Add version 2
a078c82 Add version 1
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/detached-head-practice (master)



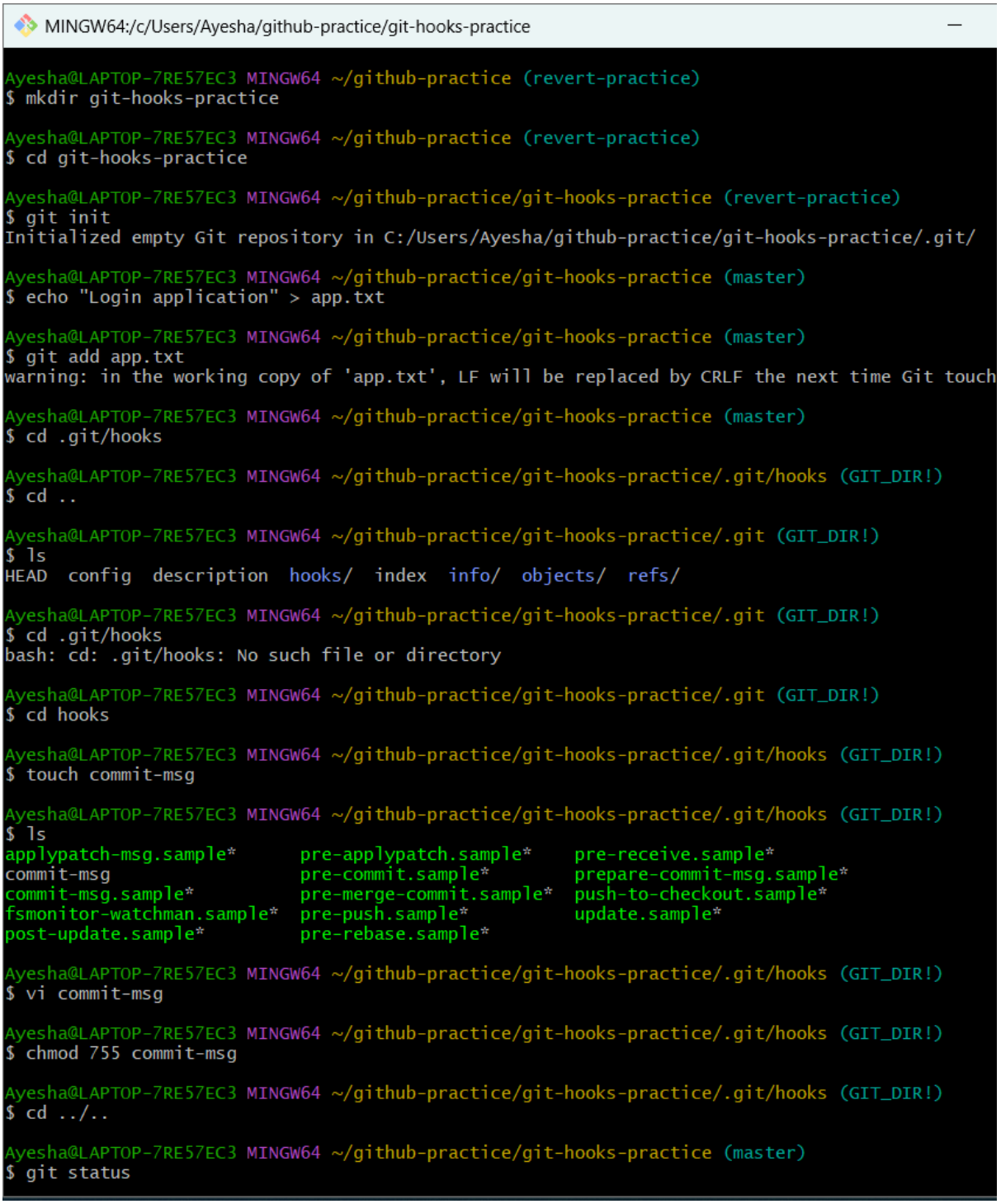
Detached HEAD

1. Create a new directory for practice.
2. Move into the new directory.
3. Initialize it as a Git repository.
4. Create a file called app.txt and add Version 1 content.
5. Stage the file.
6. Create the first commit.
7. Add Version 2 content to app.txt.
8. Stage the changes.
9. Create the second commit.
10. Add Version 3 content to app.txt.
11. Stage the changes.
12. Create the third commit.
13. Check the commit history and note the second commit ID.
14. Checkout the second commit using its commit ID.
15. Check the Git status and confirm that you are in Detached HEAD state.
16. Check the contents of app.txt and confirm that you are seeing Version 2.
17. Create a new branch from this point.

18. Switch to the newly created branch.
19. Verify that you are now on the new branch.
20. Add a new change to app.txt.
21. Stage the change.
22. Create a commit for the new change.
23. Check the commit history and verify that the new branch started from Version

11. Git Hooks Challenge

Configure a pre-commit hook to reject commits without a message format (e.g., must start with JIRA-XXX).


MINGW64:/c/Users/Ayesha/github-practice/git-hooks-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (revert-practice)
$ mkdir git-hooks-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (revert-practice)
$ cd git-hooks-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (revert-practice)
$ git init
Initialized empty Git repository in C:/Users/Ayesha/github-practice/git-hooks-practice/.git/
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ echo "Login application" > app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touch
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ cd .git/hooks
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ cd ..
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git (GIT_DIR!)
$ ls
HEAD config description hooks/ index info/ objects/ refs/
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git (GIT_DIR!)
$ cd .git/hooks
bash: cd: .git/hooks: No such file or directory
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git (GIT_DIR!)
$ cd hooks
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ touch commit-msg
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample* pre-applypatch.sample* pre-receive.sample*
commit-msg pre-commit.sample* prepare-commit-msg.sample*
commit-msg.sample* pre-merge-commit.sample* push-to-checkout.sample*
fsmonitor-watchman.sample* pre-push.sample* update.sample*
post-update.sample* pre-rebase.sample*
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ vi commit-msg
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ chmod 755 commit-msg
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ cd ../..
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ git status

```
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ ls
applypatch-msg.sample*      pre-applypatch.sample*    pre-receive.sample*
commit-msg                  pre-commit.sample*        prepare-commit-msg.sample*
commit-msg.sample*          pre-merge-commit.sample*  push-to-checkout.sample*
fsmonitor-watchman.sample*  pre-push.sample*          update.sample*
post-update.sample*         pre-rebase.sample*

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ vi commit-msg

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ chmod 755 commit-msg

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice/.git/hooks (GIT_DIR!)
$ cd ../..

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   app.txt


Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ git commit -m "Added login application"
ERROR: Commit message must start with JIRA-XXX

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ git commit -m "JIRA-123 Added login application"
[master (root-commit) eca5a09] JIRA-123 Added login application
 1 file changed, 1 insertion(+)
 create mode 100644 app.txt

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$ git log --oneline
eca5a09 (HEAD -> master) JIRA-123 Added login application

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/git-hooks-practice (master)
$
```

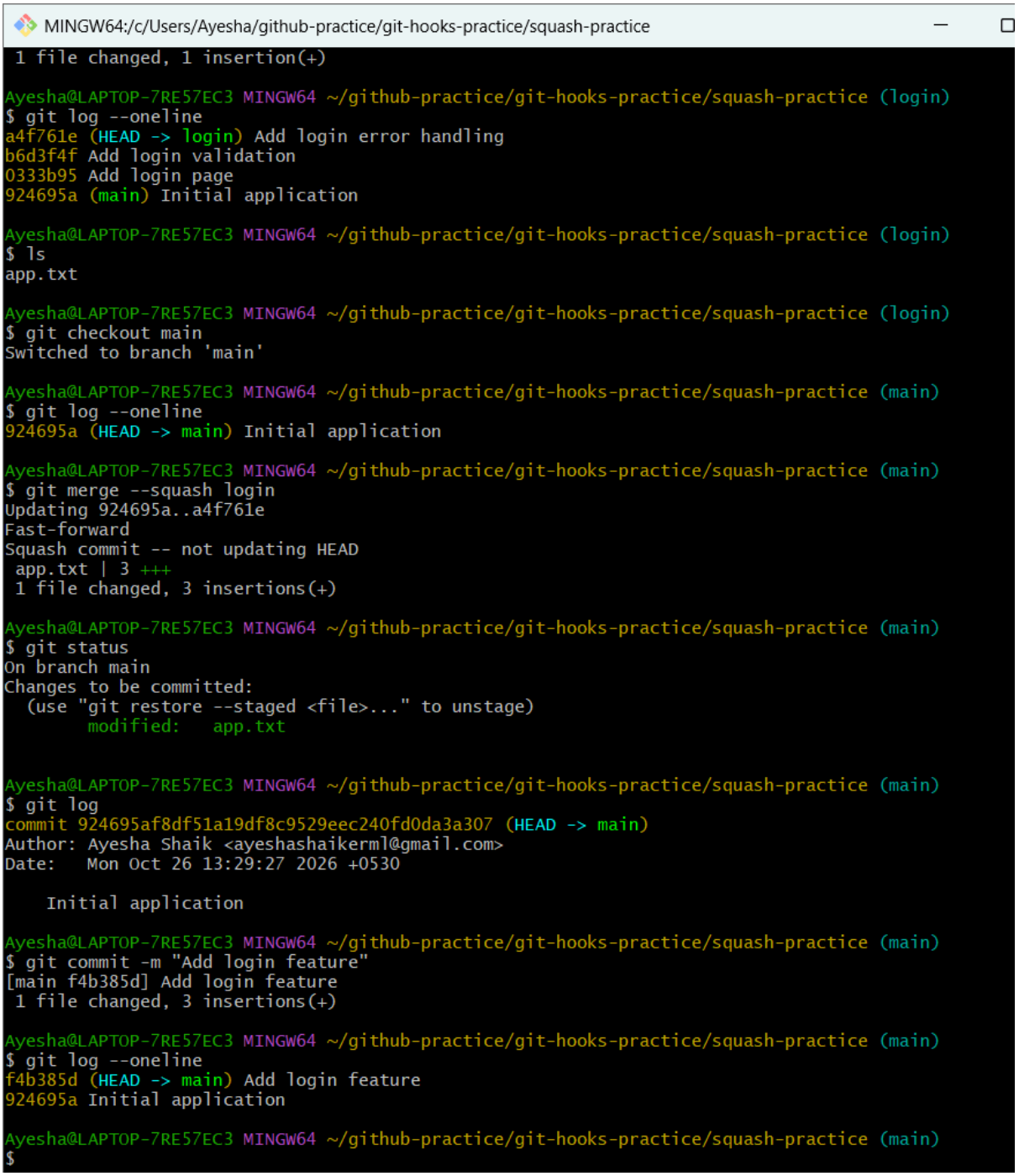
Git Hooks Challenge — Steps

1. Create a new directory for Git Hooks practice.
2. Move into the directory.
3. Initialize the directory as a Git repository.
4. Create a sample file.
5. Stage the file.
6. Go to the repository's .git/hooks directory.
7. Create a commit-msg hook.
8. Open the hook using vi.
9. Add logic to check whether the commit message starts with JIRA- followed by a ticket number.
10. If the format is invalid, make the hook reject the commit.
11. If the format is valid, allow the commit.
12. Save and exit the vi editor.
13. Make the hook executable.
14. Return to the repository directory.
15. Try committing with an invalid message.

16. Verify that Git rejects the commit.
17. Try committing with a valid message such as JIRA-123 Add login feature.
18. Verify that Git accepts the commit.
19. Check the Git history and confirm the valid commit was created.

12. Squash Merge vs Rebase Merge

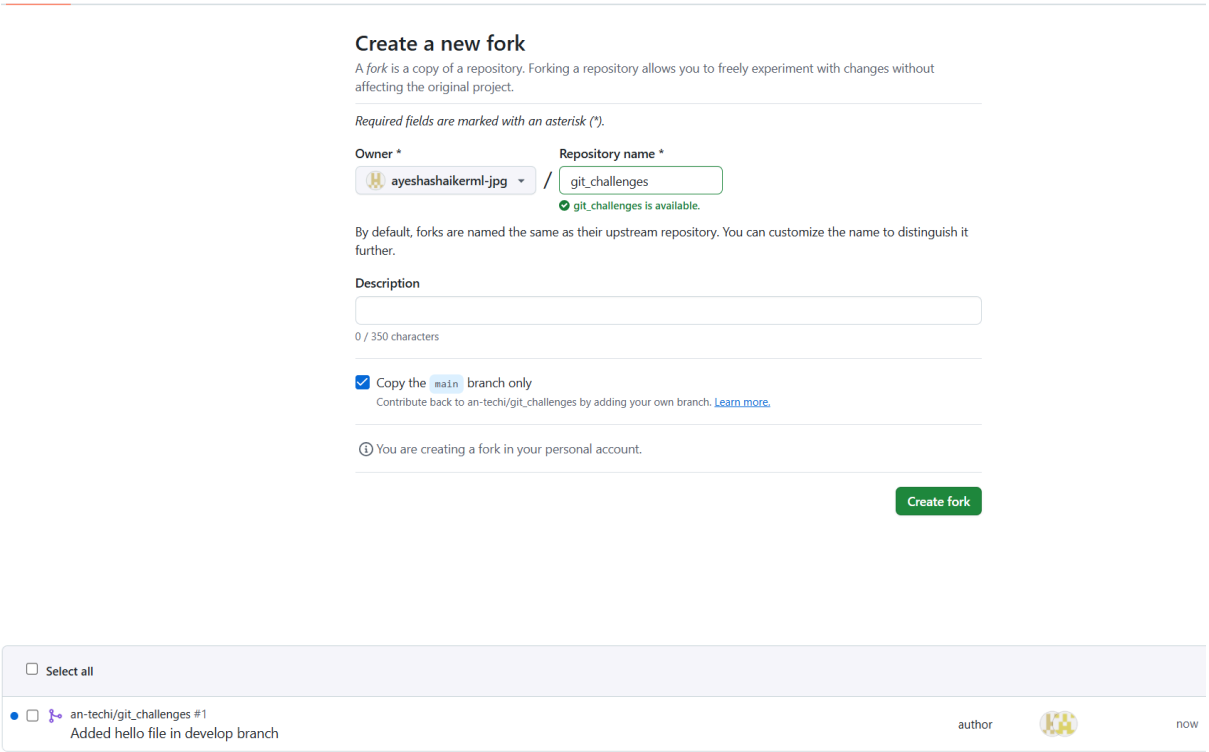
Show the difference between squash merge and rebase merge with evidence.

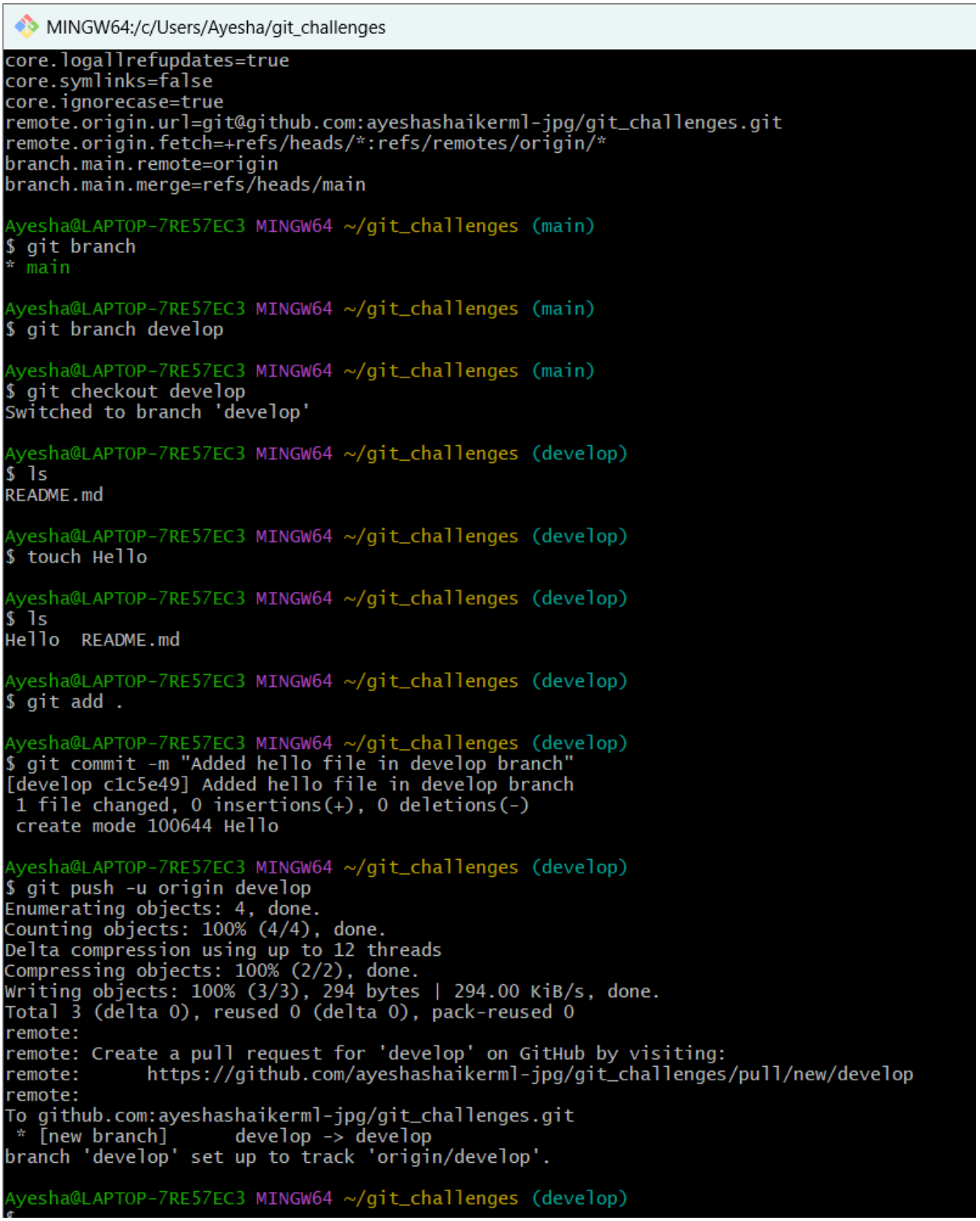


1. Create a new directory in github-practice for practising
2. Create the initial application file.
3. Make the initial commit on main.
4. Create a feature branch called login.
5. Add the login page and commit the change.
6. Add login validation and commit the change.
7. Add login error handling and commit the change.
8. Check the commit history and confirm there are 3 commits on login.
9. Switch back to main.
10. Check the main history and confirm it only has the initial commit.
11. Perform a squash merge of the login branch into main.
12. Check the status and confirm the combined changes are staged but not committed.
13. Create one new commit containing all the login changes.
14. Check the final history.
15. Confirm that main now has one login commit instead of three separate feature commits.
16. The login branch still retains its original 3 commits.

13. Fork & Pull Request Workflow

Fork a repo, make a change, and submit a pull request to the original repo.





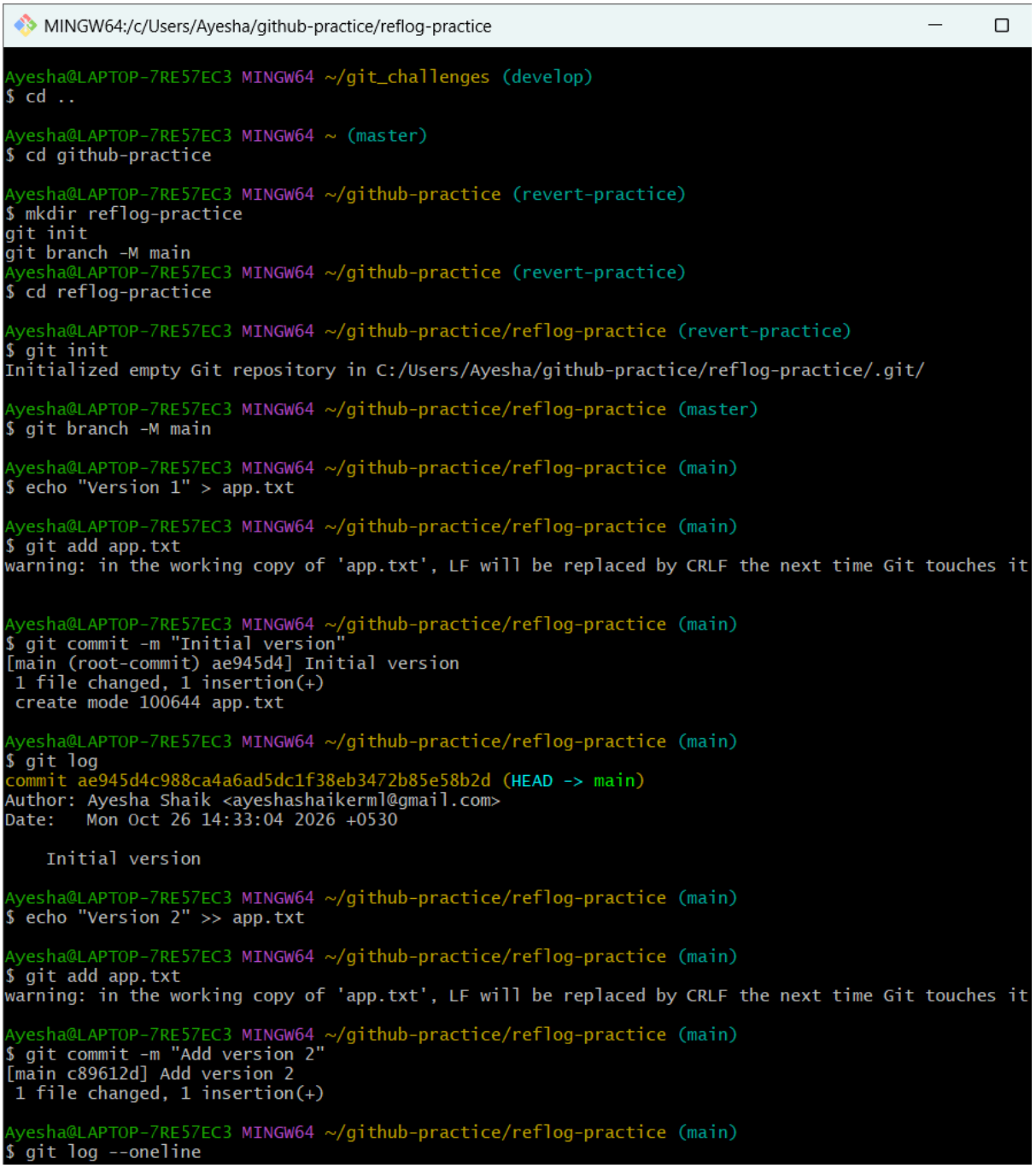
Fork & Pull Request Workflow — Steps

1. Open the original repository on GitHub.
2. Click Fork.
3. Select your GitHub account to create your own copy.
4. Open your forked repository.
5. Clone your forked repository to your local system.
6. Open the cloned repository locally.

7. Create a new branch for your changes.
8. Switch to the new branch.
9. Make the required changes to the files.
10. Review the changes.
11. Stage the changes.
12. Commit the changes with a meaningful commit message.
13. Push the new branch to your fork on GitHub.
14. Open your fork on GitHub.
15. Select the newly pushed branch.
16. Click Compare & pull request.
17. Set the base repository as the original repository.
18. Set the base branch as the original repository's target branch, usually main.
19. Set your fork and your feature branch as the head repository/branch.
20. Add a meaningful Pull Request title and description.
21. Create the Pull Request.
22. The original repository owner reviews your changes.
23. If changes are requested, modify the same branch.
24. Commit and push the additional changes.
25. The existing Pull Request is automatically updated.
26. After approval, the original repository owner merges the Pull Request.

14 Recover Lost Commit

Commit something, reset hard, and then recover it using git reflog.


MINGW64:/c/Users/Ayesha/github-practice/reflog-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/git_challenges (develop)
$ cd ..
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~ (master)
$ cd github-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (revert-practice)
$ mkdir reflog-practice
git init
git branch -M main
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice (revert-practice)
$ cd reflog-practice
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (revert-practice)
$ git init
Initialized empty Git repository in C:/Users/Ayesha/github-practice/reflog-practice/.git/
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (master)
$ git branch -M main
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ echo "Version 1" > app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git commit -m "Initial version"
[main (root-commit) ae945d4] Initial version
1 file changed, 1 insertion(+)
create mode 100644 app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git log
commit ae945d4c988ca4a6ad5dc1f38eb3472b85e58b2d (HEAD -> main)
Author: Ayesha Shaik <ayeshashaikerml@gmail.com>
Date: Mon Oct 26 14:33:04 2026 +0530
Initial version
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ echo "Version 2" >> app.txt
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git commit -m "Add version 2"
[main c89612d] Add version 2
1 file changed, 1 insertion(+)
Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git log --oneline

```
    Initial version

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ echo "Version 2" >> app.txt

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git add app.txt
warning: in the working copy of 'app.txt', LF will be replaced by CRLF the next time Git touches it

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git commit -m "Add version 2"
[main c89612d] Add version 2
 1 file changed, 1 insertion(+)

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git log --oneline
c89612d (HEAD -> main) Add version 2
ae945d4 Initial version

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git reset --hard HEAD~1
HEAD is now at ae945d4 Initial version

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git reflog
ae945d4 (HEAD -> main) HEAD@{0}: reset: moving to HEAD~1
c89612d HEAD@{1}: commit: Add version 2
ae945d4 (HEAD -> main) HEAD@{2}: commit (initial): Initial version

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git reset --hard c89612d
HEAD is now at c89612d Add version 2

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$ git log --oneline
c89612d (HEAD -> main) Add version 2
ae945d4 Initial version

Ayesha@LAPTOP-7RE57EC3 MINGW64 ~/github-practice/reflog-practice (main)
$
```

Recover Lost Commit

1. Create a new Git repository for practice.
2. Create a file and add Version 1 content.
3. Stage and commit Version 1.
4. Create another change and add Version 2 content.
5. Stage and commit Version 2.
6. Check the commit history and note the Version 2 commit ID.
7. Perform a hard reset to the previous commit.
8. Check the commit history and confirm Version 2 is no longer visible.
9. Check the file and confirm the Version 2 changes are gone.
10. Run Git reflog to view previous HEAD positions.
11. Find the Version 2 commit in the reflog.
12. Copy the commit ID.
13. Reset the branch back to the recovered commit ID.
14. Check the commit history.
15. Check the file content.
16. Confirm that Version 2 and its commit have been recovered.